

Proof Theory

Jeremy Avigad

March 4, 2014

Abstract

Proof theory began in the 1920's as a part of Hilbert's program. That program aimed to secure the foundations of mathematics by modeling infinitary mathematics with formal axiomatic systems, and proving those systems consistent using restricted, "finitary" means. The program thus viewed mathematics as a system of reasoning with precise linguistic norms, governed by rules that can be described and studied in concrete terms. Such a viewpoint, today, has applications in mathematics, computer science, and the philosophy of mathematics.

Keywords: proof theory, Hilbert's program, foundations of mathematics

1 Introduction

At the turn of the nineteenth century, mathematics exhibited a style of argumentation that was more explicitly computational than is common today. Over the course of the century, the introduction of abstract algebraic methods helped unify developments in analysis, number theory, geometry, and the theory of equations, and work by mathematicians like Dedekind, Cantor, and Hilbert towards the end of the century introduced set-theoretic language and infinitary methods that served to downplay or suppress computational content. This shift in emphasis away from calculation gave rise to concerns as to whether such methods were meaningful, or appropriate to mathematics. The discovery of paradoxes stemming from overly naive use of set-theoretic language and methods led to even more pressing concerns as to whether the modern methods were even consistent. This led to heated debates in the early twentieth century and what is sometimes called the "crisis of foundations."

In lectures presented in 1922, David Hilbert launched his *Beweistheorie*, or Proof Theory, which aimed to justify the use of modern methods and

settle the problem of foundations once and for all. This, Hilbert argued, could be achieved as follows:

- First, represent portions of the abstract, infinitary mathematical reasoning in question using formal axiomatic systems, which prescribe a fixed formal language and precise rules of inference.
- Then view proofs in these systems as finite, combinatorial objects, and prove consistency—i.e. the fact that there is no way to derive a contradiction—using unobjectionable, “concrete” arguments.

In doing so, said Hilbert,

... we move to a higher level of contemplation, from which the axioms, formulae, and proofs of the mathematical theory are themselves the objects of a contentual investigation. But for this purpose the usual contentual ideas of the mathematical theory must be replaced by formulae and rules, and imitated by formalisms. In other words, we need to have a strict formalization of the entire mathematical theory.... In this way the contentual thoughts (which of course we can never wholly do without or eliminate) are removed elsewhere—to a higher plane, as it were; and the same time it becomes possible to draw a sharp and systematic distinction in mathematics between the formulae and formal proofs on the one hand, and the contentual ideas on the other. [17]

Gödel’s second incompleteness theorem shows that any “unobjectionable” portion of mathematics is insufficient to establish its own consistency, let alone the consistency of any theory properly extending it. Although this dealt a blow to Hilbert’s program as it was originally formulated, the more general project of studying mathematical reasoning in syntactic terms, especially with respect to questions of algorithmic or otherwise concrete content, has been fruitful. Moreover, the general strategy of separating syntactic and semantic concerns and of maintaining a syntactic viewpoint where possible has become a powerful tool in formal epistemology. (See [33, 41] for more on Hilbert’s program.)

Today, Proof Theory can be viewed as the general study of formal deductive systems. Given that formal systems can be used to model a wide range of types of inference—modal, temporal, probabilistic, inductive, defeasible, deontic, and so on—work in the field is varied and diverse. Here I will focus specifically on the proof theory of *mathematical* reasoning, but even with

this restriction, the field is dauntingly broad: the 1998 *Handbook of Proof Theory* [9] runs more than 800 pages, with a name index that is almost as long as this article. As a result, I can only attempt to convey a feel for the subject’s goals and methods of analysis, and help ease the reader into the broader literature. References are generally to surveys and textbooks, and results are given without attribution.

In Section 2, I describe natural deduction and a sequent calculus for first-order logic, and state the cut-elimination theorem and some of its consequences. This is one of the field’s most fundamental results, and provides a concrete example of proof-theoretic method. In Section 3, I survey various aspects of proof-theoretic analysis, and, finally, in Section 4, I discuss some applications.

2 Natural deduction and sequent calculi

I will assume the reader is familiar with the language of first-order logic. Contemporary logic textbooks often present formal calculi for first-order logic with a long list of axioms and a few simple rules, but these are generally not very convenient for modeling deductive arguments or studying their properties. A system which fares better on both counts is given by Gerhard Gentzen’s system of *natural deduction*, a variant of which we will now consider.

Natural deduction is based on two fundamental observations. The first is that it is natural to describe the “meaning” or appropriate use of a logical connective by giving the conditions under which one can “introduce” it, that is, derive a statement in which that connective occurs, and the methods by which one can “eliminate” it, that is, draw conclusions from statements in which it occurs. For example, one can establish a conjunction $\varphi \wedge \psi$ by establishing both φ and ψ , and, conversely, if one assumes or has previously established $\varphi \wedge \psi$, one can conclude either φ or ψ , at will.

The second observation is that it is natural to model logical arguments as taking place under the context of a list of hypotheses, either implicit or explicitly stated. If Γ is a finite set of hypotheses and φ is a first-order formula, the *sequent* “ $\Gamma \Rightarrow \varphi$ ” is intended to denote that φ follows from Γ . For the most part, these hypotheses stay fixed over the course of an argument, but under certain circumstances they can be removed, or “cancelled.” For example, one typically proves an implication $\varphi \rightarrow \psi$ by temporarily assuming that φ holds and arguing that ψ follows. The introduction rule for implication thus reflects the fact that deriving ψ from a set of hypotheses Γ

together with φ is the same as deriving $\varphi \rightarrow \psi$ from Γ . Writing Γ, φ and an abbreviation for $\Gamma \cup \{\varphi\}$, the rules for natural deduction are shown in Figure 1. The quantifier rules are subject to the usual restrictions; for example, in

$\overline{\Gamma, \varphi \Rightarrow \varphi}$	
$\frac{\Gamma \Rightarrow \varphi \quad \Gamma \Rightarrow \psi}{\Gamma \Rightarrow \varphi \wedge \psi}$	$\frac{\Gamma \Rightarrow \varphi_0 \wedge \varphi_1}{\Gamma \Rightarrow \varphi_i}$
$\frac{\Gamma \Rightarrow \varphi_i}{\Gamma \Rightarrow \varphi_0 \vee \varphi_1}$	$\frac{\Gamma \Rightarrow \varphi \vee \psi \quad \Gamma, \varphi \Rightarrow \theta \quad \Gamma, \psi \Rightarrow \theta}{\Gamma \Rightarrow \theta}$
$\frac{\Gamma, \varphi \Rightarrow \psi}{\Gamma \Rightarrow \varphi \rightarrow \psi}$	$\frac{\Gamma \Rightarrow \varphi \rightarrow \psi \quad \Gamma \Rightarrow \varphi}{\Gamma \Rightarrow \psi}$
$\frac{\Gamma \Rightarrow \varphi}{\Gamma \Rightarrow \forall y \varphi[y/x]}$	$\frac{\Gamma \Rightarrow \forall x \varphi}{\Gamma \Rightarrow \varphi[t/x]}$
$\frac{\Gamma \Rightarrow \varphi[t/x]}{\Gamma \Rightarrow \exists x \varphi}$	$\frac{\Gamma \Rightarrow \exists y \varphi[y/x] \quad \Gamma, \varphi \Rightarrow \psi}{\Gamma \Rightarrow \psi}$

Figure 1: Natural deduction. A sequent “ $\Gamma \Rightarrow \varphi$ ” means that φ is a consequence of the set of hypotheses Γ . “ Γ, φ ” denotes $\Gamma \cup \{\varphi\}$.

the introduction rule for the universal quantifier, the variable x cannot be free in any hypothesis. For intuitionistic logic, one also needs the rule *ex falso sequitur quodlibet*, which allows one to conclude $\Gamma \Rightarrow \varphi$ from $\Gamma \Rightarrow \perp$, where $\perp$ represents falsity. One can then define negation, $\neg\varphi$, as $\varphi \rightarrow \perp$. For classical logic, one adds *reductio ad absurdum*, or proof by contradiction, which allows one to conclude $\Gamma \Rightarrow \varphi$ from $\Gamma, \neg\varphi \Rightarrow \perp$.

For many purposes, however, *sequent calculi* provide a more convenient representation of logical derivations. Here, sequents are of the form $\Gamma \Rightarrow \Delta$, where Γ and Δ are finite sets of formulas, with the intended meaning that the conjunction of the hypotheses in Γ implies the *disjunction* of the assertions in Δ . The rules are as shown in Figure 2. The last rule is called the “cut rule”: it is the only rule containing a formula in the hypothesis that may be entirely unrelated to the formulas in the conclusion. Proofs that do not use the cut rule are said to be cut-free. One obtains a proof system for intuitionistic logic by restricting Δ to contain at most one formula, and adding an axiomatic version of *ex falso sequitur quodlibet*: $\Gamma, \perp \Rightarrow \varphi$. The

$\overline{\Gamma, \varphi \Rightarrow \Delta, \varphi}$	
$\frac{\Gamma, \varphi_i \Rightarrow \Delta}{\Gamma, \varphi_0 \wedge \varphi_1 \Rightarrow \Delta}$	$\frac{\Gamma \Rightarrow \Delta, \varphi \quad \Gamma \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \wedge \psi}$
$\frac{\Gamma, \varphi \Rightarrow \Delta \quad \Gamma, \theta \Rightarrow \Delta}{\Gamma, \varphi \vee \theta \Rightarrow \Delta}$	$\frac{\Gamma \Rightarrow \Delta, \varphi_i}{\Gamma \Rightarrow \Delta, \varphi_0 \vee \varphi_1}$
$\frac{\Gamma, \varphi \Rightarrow \Delta \quad \Gamma, \theta \Rightarrow \Delta}{\Gamma, \varphi \rightarrow \theta \Rightarrow \Delta}$	$\frac{\Gamma, \varphi \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \rightarrow \psi}$
$\frac{\Gamma, \varphi[t/x] \Rightarrow \Delta}{\Gamma, \forall x \varphi \Rightarrow \Delta}$	$\frac{\Gamma \Rightarrow \Delta, \psi[y/x]}{\Gamma \Rightarrow \Delta, \forall x \psi}$
$\frac{\Gamma, \varphi[y/x] \Rightarrow \Delta}{\Gamma, \exists x \varphi \Rightarrow \Delta}$	$\frac{\Gamma \Rightarrow \Delta, \psi[t/x]}{\Gamma \Rightarrow \Delta, \exists x \psi}$
$\frac{\Gamma \Rightarrow \Delta, \varphi \quad \Gamma, \varphi \Rightarrow \Delta}{\Gamma \Rightarrow \Delta}$	

Figure 2: sequent calculus

cut-elimination theorem is as follows:

Theorem 2.1 *If $\Gamma \Rightarrow \Delta$ is derivable in the sequent calculus with cut, then it is derivable without cut.*

Gentzen's proof gives an explicit algorithm for removing cuts from a proof. The algorithm, unfortunately, can yield an iterated exponential increase in the size of proofs, and one can show that, in general, such an increase cannot be avoided. The advantage of having a cut-free proof is that the formulas in each sequent are “built up” directly from the formulas in the sequents above it, making it easy to extract useful information. For example, the following are two consequences of the cut-elimination theorem, easily proved by induction on cut-free proofs.

The first is known as *Herbrand's theorem*. Recall that a formula of first-order logic is said to be *existential* if it consists of a block of existential quantifiers followed by a quantifier-free formula. Similarly, a formula is said to be *universal* if it consists of a block of universal quantifiers followed by a quantifier-free formula. Herbrand's theorem says that if it is possible to prove an existential statement from some universal hypotheses, then in fact

there is an explicit sequence of terms in the language that “witness” the truth of the conclusion.

Theorem 2.2 *Suppose $\exists \vec{x} \varphi(\vec{x})$ is derivable in classical first-order logic from a set of hypotheses Γ , where φ is quantifier-free (so $\exists \vec{x} \varphi(\vec{x})$ is an existential sentence) and the sentences in Γ are universal sentences. Then there are sequences of terms $\vec{t}_1, \vec{t}_2, \dots, \vec{t}_k$ such that the disjunction $\varphi(\vec{t}_1) \vee \varphi(\vec{t}_2) \vee \dots \vee \varphi(\vec{t}_k)$ has a quantifier-free proof from instances of the sentences in Γ .*

For intuitionistic logic, one has a stronger property, known as the “explicit definability property.”

Theorem 2.3 *Suppose $\exists \vec{x} \varphi(\vec{x})$ is derivable in intuitionistic first-order logic from a set of hypotheses Γ in which neither $\vee$ nor $\exists$ occurs in a strictly positive part. Then there are terms $\vec{t}$ such that $\varphi(\vec{t})$ is also derivable from Γ .*

Theorem 2.2 provides a sense in which “explicit” information can be extracted from a certain classical proofs, and Theorem 2.3 provides a sense in which intuitionistic logic is “constructive.” We have thus already encountered some of the central themes of proof-theoretic analysis:

- Important fragments of mathematical reasoning can be captured by formal systems.
- One can study the properties of these formal systems, for example, describing transformations of formulas and proofs, translations between formulas and proofs in different systems, or “canonical” normal forms for formulas and proofs.
- The methods provide information about the logic that is independent of the choice of formal system that is used to represent it.

For more on the cut-elimination theorems, see [11, 30, 35, 37].

3 Methods and goals

3.1 Classical foundations

Recall that Hilbert’s program, broadly construed, involves representing mathematical reasoning in formal systems and then studying those formal systems as mathematical objects themselves. The first step, then, involves

finding the right formal systems. It is common today to view mathematical reasoning as consisting of a properly mathematical part that is used in conjunction with more general forms of logical reasoning, though there are still debates as to where to draw the line between the two. In any case, the following list portrays some natural systems of reasoning in increasing logical/mathematical strength:

1. pure first-order logic
2. primitive recursive arithmetic (denoted PRA)
3. first-order arithmetic (PA)
4. second-order arithmetic (PA^2)
5. higher-order arithmetic (PA^ω)
6. Zermelo-Fraenkel set theory (ZF)

Primitive recursive arithmetic was designed by Hilbert and Bernays to be a patently “finitary” system of reasoning. The system allows one to define functions on the natural numbers using a simple schema of primitive recursion, and prove facts about them using a principle of induction:

$$\varphi(0) \wedge \forall x (\varphi(x) \rightarrow \varphi(x+1)) \rightarrow \forall x \varphi(x).$$

In words, if φ holds of 0 and, whenever it holds of some number, x , it holds of $x+1$, then φ holds of every number. Here φ is assumed to be a quantifier-free formula. In fact, one can replace this axiom with a suitable induction *rule*, whereby primitive recursive arithmetic can be formulated without quantifiers at all. Surprisingly, via coding of finitary objects as natural numbers, this system is expressive and strong enough to develop most portions of mathematics that involve only finite objects and structures [3]. Peano arithmetic can be viewed as the extension of PRA with induction for all first-order formulas.

Keep in mind that there is no effective axiomatization of second- or higher-order logic that is complete for the standard semantics (where, for example, second-order quantifiers are assumed to range over all subsets of the universe of individuals). As a result, one has to distinguish axiomatic second- and higher-order logic from the corresponding semantic characterization. Axiomatically, one typically simply augments first-order logic with comprehension rules that assert that every formula defines a set (or predicate):

$$\exists X \forall y (X(y) \leftrightarrow \varphi)$$

Here φ is a formula in which X does not occur, although φ is allowed to have other free variables in addition to y . One can augment these with suitable choice principles as well. Second-order arithmetic can be viewed as the extension of Peano arithmetic with second-order logic, and second-order principles of induction, but one can, alternatively, interpret second-order arithmetic in second-order logic together with an axiom asserting the existence of an infinite domain. Similar considerations hold for higher-order logic as well.

Axioms for set theory can be found in any introductory set theory textbook, such as [22]. Of course, these axioms can be extended with stronger hypotheses, such as “large cardinal” axioms. For information on primitive recursive arithmetic, see [14, 37]; for first-order arithmetic, see [11, 15, 18]; for second-order arithmetic, see [34]; for higher-order arithmetic, see [35].

3.2 Constructive foundations

Given the history of Hilbert’s program, it should not be surprising that proof theorists have also had a strong interest in formal representations of constructive and intuitionistic reasoning. From an intuitionistic standpoint, the use of the excluded middle $\varphi \vee \neg\varphi$ is not acceptable, since, generally speaking, one may not know (or have an algorithm to determine) which disjunct holds. For example, in classical first-order arithmetic, one is allowed to assert $\varphi \vee \neg\varphi$ for a formula φ that expresses the twin primes conjecture, even though we do not know which is the case. If one restricts the underlying logic to intuitionistic logic, however, one obtains *Heyting arithmetic*, which is constructively valid.

Stronger systems tend to be based on what has come to be known as the Curry-Howard-Tait “propositions as types” isomorphism. The idea is that, from a constructive perspective, any proposition can be viewed as specifying a type of data, namely, the type of construction that warrants the claim that the proposition is true. A proof of the proposition is thus a construction of the corresponding type. For example, a proof of $\varphi \wedge \psi$ is a proof of φ paired with a proof of ψ , and so $\varphi \wedge \psi$ corresponds to the type of data consisting of pairs of type φ and ψ . Similarly, a proof of $\varphi \rightarrow \psi$ should be a procedure transforming a proof of φ into a proof of ψ , so $\varphi \rightarrow \psi$ corresponds to a type of functions. This gives rise to systems of “constructive type theory,” of which the most important examples are *Martin-Löf type theory* and an “impredicative” variant designed by Coquand and Huet, the *calculus of constructions*. Thus, our representative sample of constructive proof systems, in increasing strength, runs as follows:

1. intuitionistic first-order logic
2. primitive recursive arithmetic (*PRA*)
3. Heyting arithmetic (*HA*)
4. Martin-Löf type theory (*ML*)
5. The calculus of inductive constructions (*CIC*)

Good references for intuitionistic systems in general are [7, 38]. For more information on type theory, see [29]; for the calculus of inductive constructions in particular, see [8].

3.3 Reverse mathematics

In the 1970's, Harvey Friedman observed that by restricting the induction and comprehension principles in full axiomatic second-order arithmetic, one obtains theories that are strong enough, on the one hand, to represent significant parts of ordinary mathematics, but weak enough, on the other hand, to be amenable to proof-theoretic analysis. He then suggested “calibrating” various mathematical theorems in terms of their axiomatic strength. Whereas in ordinary (meta)mathematics, one proves theorems from axioms, Friedman noticed that it is often the case that a mathematical theorem can be used in the other direction, namely, to *prove* an underlying set-existence principle, over a weak base theory. That is, it is often the case that a theorem of mathematics is formally *equivalent* to a set comprehension principle that is used to prove it.

In that years that followed, Friedman, Stephen Simpson, and many others worked to calibrate the axiomatic assumptions used in a wide range of subjects. They isolated five key theories along the way:

1. RCA_0 : a weak “base” theory, conservative over primitive recursive arithmetic, with a principle of comprehension for recursive (computable) sets.
2. WKL_0 : adds “weak König’s lemma,” a compactness principle, to RCA_0 .
3. ACA_0 : adds comprehension for arithmetically definable sets.
4. ATR_0 : adds a principle of “arithmetical transfinite recursion,” which allows one to iterate arithmetic comprehension along countable well-orderings.

5. $\Pi_1^I - CA$: adds comprehension for Π_1^I sets.

Simpson [34] provides the best introduction to these theories and the reverse mathematics program.

3.4 Comparative analysis and reduction

We have now seen a sampling of the many formal systems that have been designed to formalize various aspects of mathematics. Proof theorists have also invested a good deal of energy in understanding the relationships between the systems. Often, results take the form of “conservation theorems” which fit the following pattern, where T_1 and T_2 are theories and Γ is a class of sentences:

Suppose T_1 proves a sentence φ , where φ is in Γ . Then T_2 proves it as well (or perhaps a certain translation, φ').

Such a result, when proved in a suitably restricted base theory, provides a foundational reduction of the theory T_1 to T_2 , justifying the principles of T_1 *relative* to T_2 . For example, such theorems can be used to reduce:

- an infinitary theory to a finitary one
- a nonconstructive theory to a constructive one
- an impredicative theory to a predicative one
- a nonstandard theory (in the sense of nonstandard analysis) to a standard one

For example:

1. Versions of primitive recursive arithmetic based on classical, intuitionistic, or quantifier-free logic, all all prove the same Π_2 theorems (in an appropriate sense) [37].
2. The Gödel-Gentzen double-negation interpretation and variations, like the Friedman-Dragalin A-translation, interpret a number of classical systems in intuitionistic ones, like PA in HA [1, 10, 12, 37, 38].
3. There are various translations between theories in the language of (first-, second-, or higher-order) arithmetic and subsystems of set theory [26, 34].

4. Both $I\Sigma_1$, the subsystem of Peano arithmetic in which induction is restricted to Σ_1 formulas, and WKL_0 , the subsystem of second-order arithmetic based on Weak König's Lemma, are conservative over primitive recursive arithmetic for the class of Π_2 sentences [2, 11, 15, 20, 32, 34, 37].
5. Cut elimination or an easy model-theoretic argument shows that a restricted second-order version, ACA_0 , of Peano arithmetic is a conservative extension of Peano arithmetic itself. Similarly, Gödel-Bernays-von Neumann set theory GBN , which has both sets and classes, is a conservative extension of Zermelo-Fraenkel set theory. See, for example, [27, 34]. In general, proofs in ACA_0 may suffer an iterated exponential increase in length when translated to PA ; and similarly for GBN and ZF , or $I\Sigma_1$ and PRA .
6. Theories of nonstandard arithmetic and analysis can be calibrated in terms of the strength of standard theories [19].
7. The axiom of choice and the continuum hypothesis are conservative extensions of set theory for Σ_1^2 sentences in the analytic hierarchy [22].

Such results draw on a variety of methods. Some can be obtained by direct translation of one theory into another. Many are proved using cut-elimination or normalization [11, 32]. The double-negation translation is a remarkably effective tool when it comes to reducing classical theories to constructive ones, and can often be supplemented by realizability, functional interpretation, or other arguments [1, 20, 36]. Model-theoretic methods can often be used, though they do not provide specific algorithms to carry out the translation [15, 18]. Even forcing methods, originally developed as a set-theoretic technique, can be fruitfully be applied in proof-theoretic settings [4, 22].

3.5 Characterizing logical strength

The results described in the previous section serve to characterize the strength of one axiomatic theory in terms of another. Showing that a theory T_2 is conservative over T_1 shows that, in particular, T_2 is consistent, if T_1 is. This provides a comparison of the *consistency strength* of the two theories.

But there are other ways of characterizing the strength of a theory. For example, the notion of an *ordinal* generalizes the notion of a counting number. Starting with the natural numbers, we can add an infinite “number,” ω , and keep going:

$$0, 1, 2, 3, \dots, \omega, \omega + 1, \omega + 2, \omega + 3, \dots$$

We can then proceed to add even more exotic numbers, like $\omega \cdot 2$, ω^2 , and ω^ω . The ordering on these particular expressions is computable, in the sense that one can write a computer program to compare any two of them. What makes them “ordinals” is that they satisfy a principle of “transfinite induction,” generalizing the principle of induction on the natural numbers. Ordinal analysis gauges the strength of a theory in terms of such computable ordinals: the stronger a theory is, the more powerful the principles of transfinite induction it can prove. See, for example, [25, 26, 35].

Alternatively, one can focus on a theory’s *computational strength*. Suppose a theory T proves a statement of the form $\forall x \exists y R(x, y)$, where x and y range over the natural numbers, and R is a computationally decidable predicate. This tells us that a computer program that, on input x , systematically searches for a y satisfying $R(x, y)$ always succeeds in finding one. Now suppose that f is a function that, on input x , returns a value that is easily computed from the least y satisfying $R(x, y)$; for example, $R(x, y)$ may assert that y codes a halting computation of a particular Turing machine on input x , and f may return the result of such a computation. Then f is a computable function, and we can say that the theory, T , proves that f is totally defined on the natural numbers. A simple diagonalization shows that no effectively axiomatized theory can prove the totality of every computable function in this way, so this suggests using the set of computable functions that the theory *can* prove to be total as a measure of its strength.

A number of theories have been analyzed in these terms. For example, by the results in the last section, the provably total computable functions of PRA , IS_1 , RCA_0 , and WKL_0 are all the primitive recursive functions. In contrast, one can characterize the provably total computable functions of PA and HA in terms of higher-type primitive recursion [5, 36], or using principles of primitive recursion along an ordinal known as ε_0 [35, 26]. Weaker theories of arithmetic can be used to characterize complexity classes like the polynomial time computable functions [11].

4 Applications

In this final section, I will describe some of the ways that proof theory interacts with other disciplines. Once again, I am only considering applications of the traditional, “metamathematical” branch of proof theory; formal deductive methods, more broadly, have applications across philosophy and the sciences, and the use of proof-theoretic methods in the study of these formal deductive systems is far too diverse to survey here.

4.1 Proof mining

One way in which traditional proof-theoretic methods have been applied is in the process of extracting useful information from ordinary mathematical proofs. The reductive results of the twentieth century showed that many classical proofs can be interpreted in constructive terms, in principle; these ideas have been adapted and extended to the analysis of ordinary mathematical proofs, in practice. Georg Kreisel described the process of extracting such information as “unwinding proofs,” and Ulrich Kohlenbach has more recently adopted the term “proof mining” [20].

Substantial work is needed to turn this vague idea into something practicable. Ordinary mathematical proofs are not presented in formal systems, so there are choices to be made in the formal modeling. In addition, the general metamathematical tools have to be tailored and adjusted to yield the information that is sought in particular domains. Thus the work requires a deep understanding of both the proof-theoretic methods and the domain of mathematics in question. The field has already had a number of successes in fields like functional analysis and ergodic theory; see, for example, [20].

4.2 Combinatorial independences

Yet another domain where a syntactic, foundational perspective is important is in the search for natural combinatorial independences, that is, natural finitary combinatorial principles that are independent of conventional mathematical methods. The Paris-Harrington statement [23] is an early example of such a principle. Since then, Harvey Friedman, in particular, has long sought to find exotic combinatorial behavior in familiar mathematical settings. Such work gives us glimpses into what goes on just beyond ordinary patterns of mathematical reasoning, and yields interesting mathematics as well. See the extensive introduction to [13] for an overview of results in this area.

4.3 Constructive mathematics and type theory

As noted above, proof theory is often linked with constructive mathematics, for historical reasons: Hilbert’s program was initially an attempt to justify mathematics with respect to methods that are finitary, which is to say, syntactic, algorithmic, and impeccably constructive. Contemporary work in constructive mathematics and type theory draws on the following facts:

- Logical constructions can often be interpreted as programming principles.
- Conversely, programming principles can be interpreted as logical constructions.
- One can thereby design (constructive) proof systems that combine aspects of both programming and proving.

The references in Section 3.2 above provide logical perspectives on constructive type theory. For a computational perspective, see [24].

4.4 Automated reasoning and formal verification

Another domain where proof-theoretic methods are of central importance is in the field of automated reasoning and formal verification. In computer science, researchers use formal methods to help verify that hardware and software are bug-free and conform to their specifications. Moreover, recent developments have shown that computational formal methods can be used to help verify the correctness of complex mathematical proofs as well. Both efforts have developed interactive approaches, whereby a user works with a “proof assistant” to construct a formal proof of the relevant claims; as well as more automated approaches, where software is supposed to carry out the task with little user input. In both cases, proof-theoretic methods are invaluable, in designing the relevant logical calculi, isolating features of proofs that enable one to cut down the search space and traverse it effectively, and replacing proof search with calculation wherever possible.

For more information on automated reasoning, see [16, 28]. For more information on formally verified mathematics, see [40], or the the December 2008 issue of the *Notices of the American Mathematical Society*, which was devoted to formal proof.

4.5 Proof complexity

Finally, the field of proof complexity combines methods and insights from proof theory and computational complexity. For example, the complexity class NP can be viewed as the class of problems for which an affirmative answer has a “short” (polynomial-size) proof in a suitable calculus. Thus the conjecture that NP is not equal to co-NP (which is weaker than saying P is not equal to NP) is equivalent to saying that in general there is no propositional calculus that has efficient proofs of every tautology. Stephen

Cook has suggested that one way of building up to the problem is to show that *particular* proof systems are not efficient, by establishing explicit lower bounds. Such information is also of interest in automated reasoning, where one wishes to have a detailed understanding on the types of problems that can be expected to have short proofs in various calculi. The works [21, 27, 31, 39] provide excellent introductory overviews.

References

- [1] Jeremy Avigad. Interpreting classical theories in constructive ones. *Journal of Symbolic Logic*, 65:1785–1812, 2000.
- [2] Jeremy Avigad. Saturated models of universal theories. *Annals of Pure and Applied Logic*, 118:219–234, 2002.
- [3] Jeremy Avigad. Number theory and elementary arithmetic. *Philosophia Mathematica*, 11:257–284, 2003.
- [4] Jeremy Avigad. Forcing in proof theory. *Bulletin of Symbolic Logic*, 10:305–333, 2004.
- [5] Jeremy Avigad and Solomon Feferman. Gödel’s functional (“Dialectica”) interpretation. In [9], pages 337–405.
- [6] *** Jon Barwise, editor. *The Handbook of Mathematical Logic*. North-Holland, Amsterdam, 1977. [Contains a number of introductory articles on proof theory and related topics.]
- [7] Michael J. Beeson. *Foundations of Constructive Mathematics*. Springer, Berlin, 1985.
- [8] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq’Art: The Calculus of Inductive Constructions*. Springer, Berlin, 2004.
- [9] *** Samuel R. Buss, editor. *The Handbook of Proof Theory*. North-Holland, Amsterdam, 1998. [Provides a definitive overview of the subject.]
- [10] Samuel R. Buss. An introduction to proof theory. In Buss [9], pages 1–78.

- [11] Samuel R. Buss. First-order proof theory of arithmetic. In Buss [9], pages 79–147
- [12] Solomon Feferman. Theories of finite type related to mathematical practice. In Barwise [6], pages 913–971.
- [13] Harvey Friedman. *Boolean Relation Theory and Incompleteness*. Cambridge University Press, to appear.
- [14] R. L. Goodstein. *Recursive Number Theory: A Development of Recursive Arithmetic in a Logic-free Equation Calculus*. North-Holland, Amsterdam, 1957.
- [15] Petr Hájek and Pavel Pudlák. *Metamathematics of First-order Arithmetic*. Springer, Berlin, 1993.
- [16] John Harrison. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press, Cambridge, 2009.
- [17] David Hilbert. Neubegründung der Mathematik. Erste Mitteilung. *Abhandlungen aus dem mathematischen Seminar der Hamburgischen Universität*, 1:157–177, 1922. Translated by William Ewald as “The new grounding of mathematics. First report.” in William Ewald, editor, *From Kant to Hilbert: A Source Book in the Foundations of Mathematics*, volume 2, Clarendon, Oxford, 1996, pages 1115–1134.
- [18] Richard Kaye. *Models of Peano Arithmetic*. Clarendon, Oxford, 1991.
- [19] H. J. Keisler. Nonstandard arithmetic and reverse mathematics. *Bulletin of Symbolic Logic*, 12:100–125, 2006.
- [20] *** Ulrich Kohlenbach. *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*. Springer-Verlag, Berlin, 2008. [An introduction to proof mining.]
- [21] Jan Krajíček. *Bounded Arithmetic, Propositional Logic, and Complexity Theory*. Cambridge University Press, Cambridge, 1995.
- [22] Kenneth Kunen. *Set Theory: An Introduction to Independence Proofs*. North-Holland, Amsterdam, 1980.
- [23] Jeff Paris and Leo Harrington. A mathematical incompleteness in Peano arithmetic. In [6], pages 1133–1142.

- [24] Benjamin Pierce. *Advanced Topics in Types and Programming Languages*. MIT Press, Cambridge, MA, 2004.
- [25] Wolfram Pohlers. Subsystems of set theory and second order number theory. In Buss [9], pages 209–335.
- [26] *** Wolfram Pohlers. *Proof Theory: The First Step into Impredicativity*. Springer, Berlin, 2009. [An introduction to ordinal analysis.]
- [27] Pavel Pudlák. The lengths of proofs. In [9], pages 547–637.
- [28] John Alan Robinson and Andrei Voronkov, editors. *Handbook of Automated Reasoning* (vols. 1 and 2). Elsevier and MIT Press, 2001.
- [29] Giovanni Sambin, editor. *Twenty-Five Years of Constructive Type Theory*. Clarendon, Oxford, 1998.
- [30] Helmut Schwichtenberg. Proof theory: Some aspects of cut-elimination. In Barwise [6], pages 867–895.
- [31] Nathan Segerlind. The complexity of propositional proofs. *Bulletin of Symbolic Logic*, 13:417–481, 2007.
- [32] Wilfried Sieg. Fragments of arithmetic. *Annals of Pure and Applied Logic*, 28:33–72, 1985.
- [33] Wilfried Sieg. Hilbert’s programs: 1917–1922. *Bulletin of Symbolic Logic*, 5:1–44, 1999.
- [34] Stephen G. Simpson. *Subsystems of Second-Order Arithmetic*. Springer, Berlin, 1999.
- [35] Gaisi Takeuti. *Proof Theory* (second edition). North-Holland, Amsterdam, 1987.
- [36] A. S. Troelstra. Realizability. In [9], pages 407–473.
- [37] *** A. S. Troelstra and Helmut Schwichtenberg. *Basic Proof Theory* (second edition). Cambridge University Press, Cambridge, 2000. [An introductory text.]
- [38] *** A. S. Troelstra and Dirk van Dalen. *Constructivism in Mathematics: An Introduction* (vols. 1 and 2). North-Holland, Amsterdam, 1988. [A helpful overview of constructive mathematics.]

- [39] Alasdair Urquhart. The complexity of propositional proofs. *Bulletin of Symbolic Logic*, 1:425–467, 1995.
- [40] Freek Wiedijk. *The Seventeen Provers of the World*. Springer, Berlin, 2006.
- [41] *** Richard Zach. *Hilbert’s program then and now*. In Dale Jacquette, ed., *Philosophy of Logic*, Elsevier, Amsterdam, 2006, pages 411–447. [A nice historical overview.]